

Figure 6: Failed test case



Figure 7: Actions

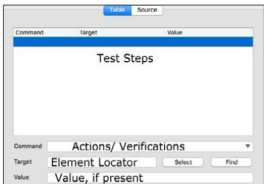


Figure 8: Parameters for assessment/ assertion



Figure 9: Parameters for assessors

There are multiple options available to save the test suite or the test case in various formats like Ruby RSpec, Ruby Test::Unit, Java JUnit, Java TestNG or C# NUnit.

Pillars of the Selenium IDE

Now let's understand the functions we just used in the example. There are three pillars that support the Selenium IDE.

1. **Action:** This refers to the operations

Well-known extensions

Name	Purpose
Favourites	To mark a test suite as favourite and execute it in one click
Flex Pilot X	For Flex based automation
FlexMonkium	For Adobe Flex based recording and playback testing in Selenium IDE
File Logging	For saving logs in a file
Flow Control	To control the test execution flow
Highlight Elements	To highlight a Web control
Implicit Wait	To wait for an element for a certain time limit
ScreenShot on Fail	To take a screenshot on failure
Test Results	To save the test case result for a test suite, in one click

that are being performed with the UI.

2. **Assessors/Assertion:** This helps in verification of the data we get from the UI.

3. **Locator strategy:** This helps to find the element that will perform the operation, from the UI.

Presently, the Selenium IDE has an exceptionally developed library with a lot of actions, assertion/assessors and locator strategies.

Creating user-defined functions for test cases - user extensions

Typically, every once in a while, we have to add some more features for our venture. In such circumstances, we can grow this library by including custom extensions, called 'user extensions'.

Let's assume we require an action that can capitalise the content before filling it in a Web component. We don't have any action to capitalise the content in the default action library. In this case, we can make our own 'user extension'. In this instructional exercise, let's figure out how to make a user extension to change text to upper case.

Need for user extensions

To generate the user extension, one needs to learn the basics of JavaScript. We have

to generate JavaScript functions, and add these to the Selenium object prototype and PageBot object prototype to create their own user extensions.

How does the Selenium IDE recognise the user extension?

After adding the user extension to the Selenium IDE, when we open it, all the extensions in the JavaScript prototype get loaded and the Selenium IDE recognises them by their name.

Creating a user extension

Step 1 - Action: All actions are to begin with 'do'; for example, in our case, the action could be `doTextUpperCase`. The Selenium IDE will automatically generate the wait method when we include the extension. So, for this situation, when we initiate the `doTextUpperCase` action, the IDE will generate a corresponding wait method as `TextUpperCaseAndWait`, which will accept two arguments.

Step 2 - Assessors/Assertion: All assessors are prefixed by 'get' or 'is'; for example, `getValueFromCompoundTable` and `isValueFromCompoundTable`. Two arguments, one for the target field and another for the value field of the test case, are required.

For each assessor, there will be the corresponding verification functions prefixed by 'verify' and 'assert' and the wait function prefixed by 'waitFor'.

Step 3 - Locator strategy: If we wish to generate our own method to locate an element, then we need